From Symptom to Structure: Analyzing Disagreement in Agent-Skill Security Scanners

Abstract

Automated security analyses routinely disagree, yet evaluations treat that disagreement as a statistic—Cohen's κ, pairwise overlap—and stop there, as if it were noise or merely an outcome about tool quality. We propose to treat disagreement differently: as *analytical evidence about the analyses themselves*. The structure of disagreement, once made comparable, exposes each analysis's coverage of the threat space and how it operationalizes detection there. We develop a methodology for this conversion: a shared observation language that makes disagreement localizable, a decomposition into *coverage gaps* and *operational divergence*, blind-spot hypotheses locked before construction, and adversarial validation. We instantiate the idea on agent-skill security scanners, an emerging supply-chain defense whose scanners disagree sharply. Across 582 real malicious skills and three heterogeneous scanners, the decomposition predicts concrete blind spots, and locked adversarial construction confirms them: hidden-file payloads evade SkillSpector on 14/15, expression variants evade Cisco on 6/6, and combined strategies evade SkillSpector on 5/5, with selected failures traced to source-level mechanisms. Disagreement, we argue, is not merely a statistic to report: it is evidence to mine.

Keywords

agent skills, security scanners, taxonomy, blind spots, adversarial construction

1 Introduction

Automated security analyses disagree. Given the same artifact, one scanner flags it, another passes it, and a third flags something else entirely. The standard response is to *measure* the disagreement—Cohen's κ, pairwise overlap, per-tool recall—and treat the result as an evaluation outcome: a fact about tool quality, or noise to be averaged away. We ask a different question: *what does the pattern of disagreement reveal about the analyses themselves?* We argue that disagreement is not merely a statistic to report. It is analytical evidence: once disagreements are made comparable, their structure exposes what each analysis covers, what it cannot see, and why.

Today this evidence is inaccessible, because analyses speak incompatible languages. One scanner reports "tool poisoning," another "supply-chain risk," a third a family of regex matches. There is no shared frame in which to ask *where exactly* they disagree, so disagreement remains global—a κ—but never localizable. Without localization there is no explanation; without explanation, disagreement cannot be acted upon.

We propose to convert disagreement into structure. The conversion has four steps. First, a *shared observation language*—a coordinate space over the threat space—into which each scanner's categories are mapped, making disagreement localizable. Second, a *structural decomposition*: localized disagreement separates into *coverage gaps* (a scanner does not inspect a region at all) and *operational divergence* (scanners cover the same region but instantiate detection differently). Third, the decomposition yields *blind-spot hypotheses*, which we lock before building anything. Fourth, *adversarial validation*: coordinate-driven construction tests the locked hypotheses, and selected failures are traced to implementation mechanisms.

We instantiate this idea on *agent-skill security scanners*. Coding agents extend their capabilities by installing skills—packages of instructions and scripts—and malicious skills are a live supply-chain threat (the ClawHavoc incident surfaced over 1,184 of them [6]). Scanners have emerged to screen skills, but they disagree sharply [2, 6], and no existing evaluation explains the disagreement. The object is timely; the methodology is the contribution.

An honesty point frames our results. In aggregate the scanners are strong: across 582 real malicious skills, SkillSpector detects 92.8%, Cisco 85.2%, Caterpillar 72.5%. The blind spots are therefore invisible in average recall; they emerge only when the threat space is partitioned. This is precisely the structure that aggregate metrics conceal, and precisely what disagreement, treated as evidence, exposes.

Contributions. This is an idea paper: one general claim, a methodology, an instantiation, and early evidence.

- **Idea.** Disagreement among automated security analyses is analytical evidence, not merely an evaluation outcome.
- **Methodology.** A conversion pipeline: shared observation language → structural decomposition → locked hypotheses → adversarial validation (§3–§6).
- **Instantiation.** Agent-skill security scanners, studied via a three-dimensional attack-coordinate space.
- **Early evidence.** 582 real malicious skills, three scanners, 129 locked adversarial constructions, and source-level mechanism tracing.

Scope. We demonstrate the loop end-to-end on one instantiation. The generality of the idea—to package scanners, dependency scanners, static analyzers, LLM-based security evaluators—is the research direction this early result opens, to be developed in an extended version.

Results. Hidden-file payloads evade SkillSpector on 14/15 samples (its build context skips dotfiles). Expression-layer variants evade Cisco on 6/6 (its pipeline requires literal command surfaces). Combined strategies evade SkillSpector on 5/5. A regex-only reference tool is blind to all 10 semantic variants, showing blind spots deepen as detection grows more primitive.

Paper identity. Disagreement is not merely a number: it reveals structure that can be hypothesized about and tested.

2 Background and Motivation

Agent skills are distributed through marketplaces such as ClawHub and skills.sh [6]; at scale, a snapshot of 138,133 public SKILL.md files found 91.8% exhibiting at least one packaging or safety defect [11], and field measurements attribute 26.1% of 31,132 wild skills with at least one security vulnerability [7]. Because a skill can

Table 1: The three studied scanners. "Decision rule" is what the raw output reduces to for an allow/block decision: each rule is later implicated in a distinct blind spot.

Scanner	Layers	Decision rule
SkillSpector	static rules, AST, YARA, LLM semantic	risk score (0–100); >50 = detected
Cisco	static, YARA, LLM judgment, dataflow	safe unless a CRITICAL/HIGH finding exists
Caterpillar	regex families	letter grade; ≠A = flagged

carry arbitrary instructions and scripts, it is an active supply-chain attack surface: the ClawHavoc incident identified over 1,184 malicious skills in a single marketplace [6], and injected skill instructions achieve up to 80% attack success on frontier agents [8]. Security scanners have emerged to screen skills before installation [2].

We study three scanners spanning the mainstream detection architectures (Table 1). **SkillSpector** (NVIDIA) combines static rules, AST analysis, YARA signatures, and an LLM semantic layer. **Cisco Skill Scanner** (AI Defense) combines static analysis, YARA, LLM-based judgment, and behavioral dataflow. **Caterpillar** is a lightweight regex-based tool. SkillSpector and Cisco are production-grade; Caterpillar serves as a lower-bound reference showing how blind spots deepen as detection grows more primitive. The three decision rules already preview the operational divergence that §4 measures: a numeric risk score, a severity threshold, and a lexical grade.

Threat model. The attacker authors and publishes a skill that a user (or an agent acting for the user) installs from a marketplace. The skill may contain natural-language instructions in SKILL.md, arbitrary scripts, and metadata declarations; everything is readable, but nothing is executed at install time. The defender runs one or more scanners over the skill *before* execution and must decide allow/block with no runtime evidence. The attacker's goal is not to exploit a parser bug but to place malicious behavior where the defender's analysis does not look: outside the scanned file set, outside the literal command surface, or inside a declared capability. All blind spots we report are of this placement kind: architectural, not cryptographic.

These scanners frequently disagree on the same skill. Recent measurements quantify the disagreement: across 67,453 skills, pairwise overlap of flagged sets is at most 10.4%, and only 0.69% are flagged by all scanners [6]. Yet such numbers stay descriptive: they tell us *that* scanners disagree, not *where* in the attack space the disagreement arises or *why*.

3 A Shared Observation Language

The reason disagreement cannot currently be interrogated is that the scanners speak incompatible languages: SkillSpector speaks of 17 risk classes, Cisco of 18 threat categories, Caterpillar of regex rule families. There is no frame in which to ask *where exactly* they disagree. The coordinate space we introduce is, first and foremost, a *shared observation language*: the instrument that makes disagreement localizable. It is not a universal taxonomy; it is the observation layer required to interrogate disagreement. Figure 1 shows the full conversion this paper instantiates.

Table 2: The three-dimensional observation language. Each threat is a coordinate $(source, mechanism, target)$; all three axes are multi-valued.

Dimension	Question	Values (abbreviated)
source	where does it enter?	supply chain; user input; external untrusted content; runtime environment; unknown
mechanism	how does it attack?	code execution; instruction manipulation; dependency manipulation; privilege abuse; obfuscation; state corruption; …
target	what does it seek?	information theft; persistent control; resource abuse; system damage; financial theft; content-safety harm; defense evasion

3.1 Three Dimensions

Each threat is located by three dimensions (Table 2): *source* (where it enters: supply chain, user input, external untrusted content, runtime environment, or unknown), *mechanism* (how it attacks: code execution, instruction manipulation, dependency manipulation, privilege abuse, obfuscation, state corruption, …), and *target* (what it seeks: information theft, persistent control, resource abuse, system damage, financial theft, content-safety harm, defense evasion). The axes follow established security modeling: the attacker model (cf. Dolev–Yao), the technique axis (cf. ATT&CK), and the security-property axis (cf. the CIA triad).

3.2 Mapping Scanner Languages

We mapped 67 scanner-native categories into the space, producing 43 coordinates. The mapping is manual and evidence-based: each native category (e.g. Cisco's "tool poisoning," SkillSpector's 17 risk classes, Caterpillar's rule families) is assigned to the set of coordinates its triggering conditions actually observe, with multi-valued assignment when a category spans several coordinates; a credential-harvesting rule, for instance, covers any (source × code execution × information theft) cell regardless of how the credential was reached. This is where "semantic alignment" becomes checkable: two categories from different scanners that map to overlapping coordinate sets are nominally the same detector, and any residual disagreement between them is operational, not semantic. To check that the space can carry existing threat descriptions, we mapped 1,253 threat statements from 84 sources. Only 17.9% map cleanly to a full triple; **56.5% are partial triples**: existing vocabularies cannot even fully express them, and 20.3% fall into genuine gaps. This partial-triple rate is our key measurement-space evidence: current categorizations do not provide a complete description; a coordinate space does.

3.3 Orthogonality

The dimensions carry complementary information. On an 81-row human-verified gold standard, pairwise normalized mutual information is 0.565 (source×mechanism), 0.492 (mechanism×target), and 0.232 (source×target): the axes correlate but do not collapse into each other. Ablating any single axis destroys 21% (source),

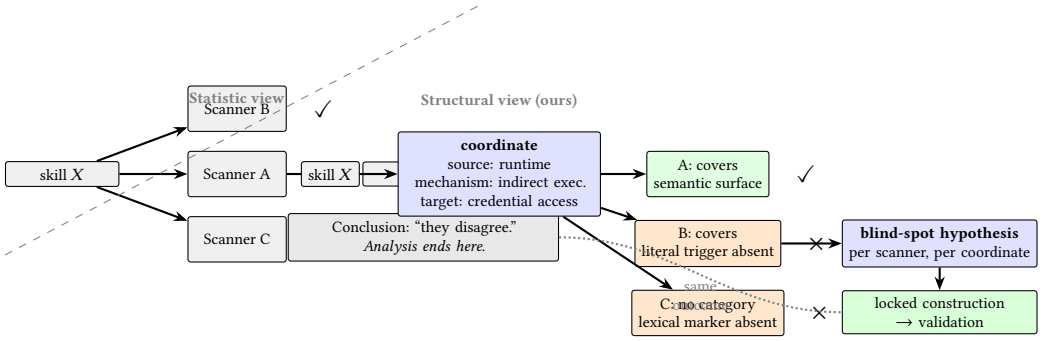

Figure 1: The same disagreement, read two ways. The statistic view collapses three verdicts on skill X into a scalar and stops. The structural view localizes skill X to an attack coordinate, reads each scanner's coverage and detection surface there, derives per-scanner blind-spot hypotheses, and validates them by locked construction. Same outcome, different explanatory power.

38% (target), or 41% (mechanism) of coordinate resolution, and full three-dimensional coordinates resolve threats 2.6× more finely than the best pair. The space therefore carries non-redundant information and serves as the observation layer for all subsequent analysis.

4 Structural Disagreement Analysis

With a common observation layer in place, we project each scanner's detections onto the coordinate space and compare scanners coordinate by coordinate. This is the paper's core analysis, measured on 582 real malicious skills (350 wild samples stratified by behavior, 232 coordinate-generated).

4.1 Aggregate Detection Conceals the Structure

Table 3 shows what the aggregate view sees—and what it hides. In aggregate the scanners are strong: SkillSpector detects 92.8%, Cisco 85.2%, Caterpillar 72.5%. Stopping here, one would conclude detection is largely solved. Yet the per-sample flag distribution shows 39.2% of malicious skills are missed by at least one scanner, and the *combination* column shows the misses are not random: Caterpillar alone accounts for the largest missed group (C+S, 118), Cisco is the sole miss in 45, SkillSpector in only 16. The blind spots are invisible at this level of abstraction; they emerge only when the attack space is partitioned.

4.2 Coverage Gaps

The first structural cause of disagreement is a *coverage gap*: a scanner does not inspect a coordinate at all. The per-dimension agreement pattern (Table 4) localizes where. This analysis runs on the finding-level corpus from our taxonomy-mapping stage (three scanners with declarable category systems that surfaced shared findings: SkillSpector, Snyk, and ATH), since κ requires comparable per-finding labels. After mapping those findings onto coordinates, agreement splits by axis. The pre-taxonomy baseline is near-zero ($\kappa \approx 0.13$ on shared binary flags). After mapping, agreement on *target* rises to +0.223—the shared language unifies what an attack wants—while *mechanism* stays near zero (−0.025) and *source* turns negative (−0.137): scanners report behavior, not provenance, and two of them read provenance in systematically opposite ways (Snyk external-content vs. ATH supply-chain, $\kappa = -0.395$ on shared findings). The split is the point: one third of the disagreement is a language problem the coordinate space solves, and the rest is structural, which is what the decomposition next isolates.

Coordinates whose source is the runtime environment or user input are systematically less covered than supply-chain coordinates: a payload whose malice lives in a runtime dataflow (user input interpolated into a command pipeline) has no static surface in the skill text, and scanners that do not model that flow cannot see it at all.

Table 3: Observed disagreement on 582 real malicious skills (350 wild + 232 coordinate-generated). Readings recover what each product's shipped decision rule discards: Cisco counts MEDIUM-or-higher (recovers 80 threshold-swallowed findings), SkillSpector any nonzero risk score, Caterpillar any finding. Completion: Cisco 564/582, SkillSpector 541/582, Caterpillar 582/582 (scanner-side failures excluded).

(a) Detection rate by subset

Subset (n)	Cisco	SkillSpect.	Caterp.
Wild (350)	87.4%	91.1%	70.9%
Generated (232)	82.3%	95.3%	75.0%
All (582)	85.2%	92.8%	72.5%

(b) How many scanners flag a skill

Flags	n	share
3 (all)	354	60.8%
2	179	30.8%
1	39	6.7%
0	10	1.7%

(c) Pairwise consensus (third scanner is the sole miss)

Pair flags	n	sole miss
Cisco + SkillSpector	118	Caterpillar
SkillSpector + Caterpillar	45	Cisco
Cisco + Caterpillar	16	SkillSpector
Single-scanner flags	39	SS 23 / Cisco 8 / Cat 7

Table 4: Inter-scanner agreement by dimension, on 793 deduplicated shared findings across SkillSpector, Snyk, and ATH (finding-level Cohen's κ; pairs evaluated on skills both scanners flag). The pre-taxonomy baseline on shared binary flags is $\kappa \approx 0.13$.

Dimension	mean κ	best pair	worst pair
target	+0.223	SS×Snyk 0.550	SS×ATH −0.130
mechanism	−0.025	SS×ATH 0.131	Snyk×ATH −0.222
source	−0.137	SS×ATH −0.015	Snyk×ATH −0.395

4.3 Operational Divergence

The second cause is subtler. Scanners can *semantically agree* that a coordinate matters while *operationally diverging* on how to detect it. Consider the coordinate (credential access × code execution × data exfiltration): all three scanners nominally cover it, yet SkillSpector instantiates detection through script-level semantic evidence, Cisco through literal command triggers, and Caterpillar through lexical regex patterns. Same coordinate, same semantic intent, three different operational surfaces. An attack that removes the literal surface evades Cisco and Caterpillar while SkillSpector may still catch it, and vice versa for attacks that hide the script.

Figure 2 shows where the two production scanners' declared coverage diverges, coordinate by coordinate. The map separates two kinds of disagreement that a κ conflates. In the 10 *both-declare* coordinates, disagreement is operational: both scanners cover the region but instantiate detection differently, SkillSpector via script-level semantic evidence and Cisco via literal command triggers. An attack that removes the literal surface evades Cisco while SkillSpector may still catch it; attacks that hide the script do the reverse. In the 16+4 *single-coverage* coordinates, the disagreement is a coverage gap: one scanner simply has no category that maps there. The 13 *neither* coordinates are collectively unguarded by both production scanners; and the regex tool is structurally unmappable—it has no category system at all. Construction results confirm the reading: on the both-declare region, pure-instruction runtime payloads drop Cisco to 22% while SkillSpector holds 100%; hidden-file payloads drop SkillSpector to 27% while remaining visible to YARA-level rules (a threshold failure, not a gap; §6); removing literal commands blinds the regex engine entirely (0/10) while SkillSpector is unaffected. No attack form is missed by all scanners, but no scanner survives all forms. Disagreement of this shape is not noise; it is a map of each scanner's operational surface.

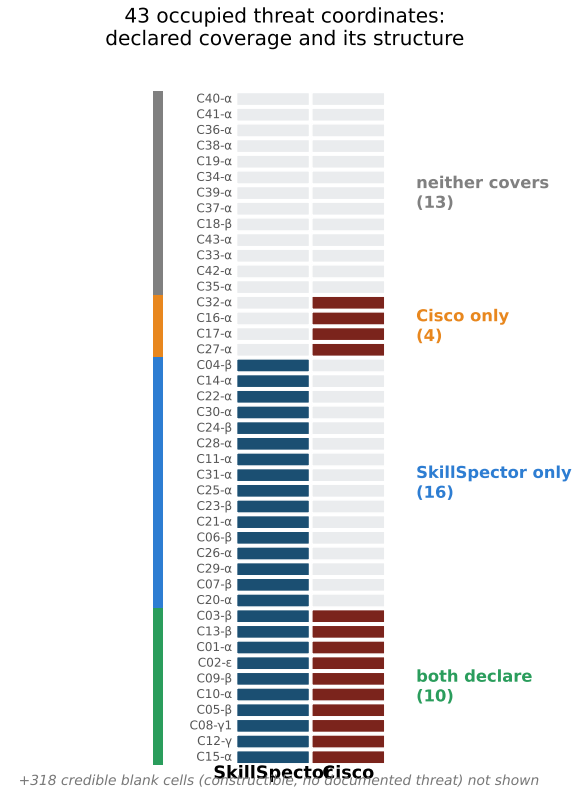

Figure 2: The structure of declared coverage across 43 occupied threat coordinates. Rows are coordinates (IDs at left); filled cells are categories the scanner maps onto that coordinate. The four bands are four disagreement states: 10 coordinates where both production scanners declare coverage (candidate operational divergence), 16 covered only by SkillSpector, 4 only by Cisco, and 13 covered by neither (several only by external reference taxonomies). A regex tool with no category system appears in no column; it cannot be mapped at all. Color encodes *disagreement type*, not detection rate.

4.4 Misclassification and the Grey Zone

A third, orthogonal finding: scanners can detect a behavior but misclassify its intent. In a pilot audit of 30 SkillSpector findings, every one described a real capability that was in fact a legitimate, declared tool behavior: "dangerous but legal." Detection fired; the verdict was wrong. This is the false-positive side of the same structural story: the scanners' vocabularies have no slot for "declared, justified capability," so a skill that legitimately reads credentials for its stated purpose lands in the same category as one that exfiltrates them. For governance, an unchecked false-positive rate is as corrosive as a blind spot: users disable scanners that cry wolf, and the blind spot then inherits the whole pipeline. Our decomposition treats the grey zone as a first-class object: the misclassification rate is a property of the operational divergence, not of the corpus.

5 From Analysis to Blind-Spot Hypotheses

The coordinate space is not itself a blind-spot oracle. It is a common observation layer through which two kinds of evidence are turned into explicit, pre-specified hypotheses, which we lock *before* building any adversarial sample. To prevent post-hoc rationalization, we separate exploratory experiments from confirmatory ones.

5.1 Two Evidence Sources

Type A (coordinate-coverage evidence). Projecting every mapped category onto the space yields a scanner-by-coordinate coverage matrix. The matrix makes absence explicit: of the 400 cells in the cross-product of our observed dimension values, only 43 are occupied by any documented threat, 318 are credible blanks (constructible but undocumented), and 39 are infeasible. A coordinate uncovered or weakly covered by a scanner's mapped categories yields the hypothesis that the scanner will miss attacks occupying that coordinate, and the 318 credible blanks delimit exactly where such hypotheses can be tested.

Type B (scanner-architecture evidence). An implementation property produces a mechanistic limitation. Reading SkillSpector's source, for instance, shows that its build context skips dotfiles; this produces the hypothesis that hidden-file payloads will evade analysis.

5.2 Sanitization: Separating Real Misses from Artifacts

Construction experiments are easily polluted by artifacts that look like scanner misses. Our protocol applies three checks to every sample. *Malice realism:* each sample is verified to contain the attack it was specified to contain: LLM generators sometimes emit a benign variant or silently drop the payload script, and a "miss" on such a sample is a generator artifact. *Technical-failure classification:* scanner API timeouts, malformed submissions, and crashes are classified as indeterminate and excluded, never counted as misses. *Ground-truth hygiene:* provenance metadata and experiment-arm prefixes are stripped before scanning, so no classifier can leak the label through the filename. The discipline is load-bearing: an early construction round appeared to show a 56% all-scanner miss rate; after sanitization the true all-miss count was zero. Without these checks, a construction experiment measures the generator, not the scanner.

5.3 Hypothesis-Locking Protocol

For each hypothesis we record the evidence, the locked prediction, the construction specification, and the validation outcome. Locking precedes construction: the specification is written from the hypothesis, not from observed misses. Our timeline is explicit and reproducible: the coordinate-coverage matrix and blind-spot hypotheses were recorded on 2026-08-13; confirmatory construction began 2026-08-19. The hidden-file hypothesis, for instance, was derived by reading SkillSpector's build-context source *before* any hidden-file sample existed. Arms 1–6 preceded the matrix and are labeled exploratory: they establish that coordinate-driven construction is feasible, and none of their outcomes enters the hypothesis table.

6 Early Evidence: Adversarial Validation

We now demonstrate that the loop works end-to-end. This is early evidence, proof of feasibility for the idea and the methodology, rather than a claim of benchmark completeness. We validate locked hypotheses through coordinate-driven adversarial construction: 129 constructed samples across 13 experiment arms, each scanned by all three scanners.

6.1 Centerpiece: Hidden-File Payloads

Evidence (Type B): SkillSpector's build context skips dotfiles. Figure 3 traces one payload through all three scanners. *Locked hypothesis:* payloads placed in hidden files (`.env`, `.profile`) will evade SkillSpector. *Construction:* 15 skills whose SKILL.md is benign, the payload living entirely in a dotfile. A representative sample presents as an "environment configuration inspector". The SKILL.md instructs the agent to validate configuration hygiene. The bundled `.env` (a shell script despite its name) collects cloud keys and tokens, base64-encodes them, and exfiltrates them via `curl` under a browser User-Agent to an attacker endpoint. *Outcome:* **14/15 evade** SkillSpector (score ≤50); 11 score exactly 0. The mechanism is confirmed at source level: the scanner's raw output records the analyzed components as exactly `['SKILL.md']`: the dotfiles never entered the scanned context. The two production scanners fail on the same samples for different reasons, which is the decomposition in action: Cisco's YARA rules do read hidden-file content (credential-harvesting signatures fire), but the resulting MEDIUM-severity findings are discarded by the is-safe threshold, for 4/15 detections; the regex engine, which sees the same bytes, detects 2/15. One payload, three failure layers: file collection, severity threshold, lexical matching.

6.2 Role of the Exploratory Arms

Arms 1–6 preceded hypothesis locking and serve two feasibility roles only. First, they established that the coordinate space drives construction: Arm 1 generated one skill per 24 documented attack primitives and showed that detectability tracks the *primitive* (direct `eval` is caught everywhere; indirect `compile`/`getattr` construction is missed by Cisco), not the coordinate, an early sign that operational surfaces, not coordinates, decide detection. Second, they calibrated the generator: Arm 2 showed that semantic re-wrapping of the description layer leaves SkillSpector detection unchanged (its evidence comes from code), and Arm 5 showed that *false* capability declarations are counterproductive (the description-code contradiction detector fires on them). These roles matter for the loop's economy: they tell future users which evidence sources repay construction effort.

6.3 Confirming Cases

Table 5 summarizes the remaining locked hypotheses: the evidence, the locked hypothesis, the construction, and the outcome. Three deserve mechanism notes. The expression-variant construction (0/6 on Cisco) builds the same code-execution behavior through indirect construction: `compile()` of a string, `getattr` reflection, dynamic `__import__`, so the YARA/DFA layers find no literal command surface and the LLM layer, given no high-severity static finding to anchor on, defers; the same samples remain fully visible to SkillSpector, which reasons over script semantics. The no-literal construction (0/10 on the regex tool) expresses exfiltration as tool semantics ("archive the result to the remote repository") with no `curl`/`base64` token anywhere; the regex engine has literally nothing to match. The combined-strategy construction (5/5 partial bypass on SkillSpector) stacks independent weak mechanisms (hidden placement, indirect execution, and declared-capability cover), each individually below the detection threshold, and the scoring

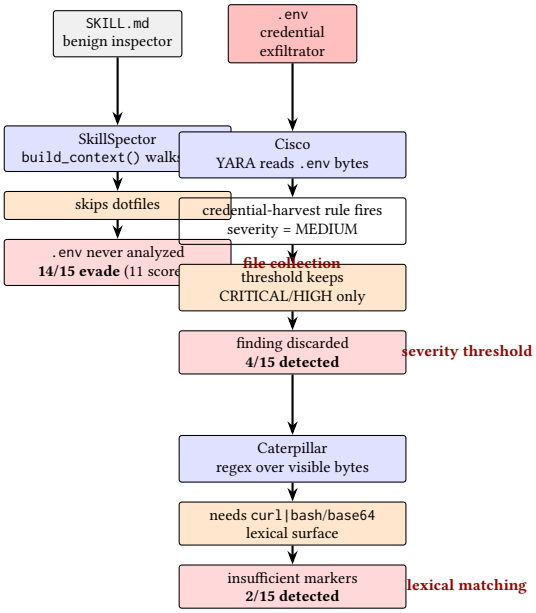

Figure 3: One payload, three failure mechanisms. The same hidden-file skill produces misses (or discarded detections) in all three scanners for three different architectural reasons: SkillSpector's file collection never reads the dotfile, Cisco's YARA layer sees it but the is-safe threshold discards the MEDIUM finding, and the regex engine lacks the lexical surface its rules require. Same result, three distinct blind spots, which is exactly what a scalar κ cannot express.

layer, which aggregates isolated checks, never sees the conjunction.

6.4 Honest Scoping

Two caveats bound our claims. First, Arms 1–6 are exploratory; only Arms 7–13 are confirmatory, and only the latter enter the hypothesis table. Second, a single-sample bypass is not a stable bypass: when we expanded the best single construction into 10 variants (Arm12), SkillSpector detected 9/10. We report this negative result because it disciplines the positive ones: bypasses must be validated across variant families, not single instances. Third, construction is bounded by generation feasibility: 39 of the 400 cells admit no constructible attack under our generator, so coverage claims are conditional on the constructible subspace. What the loop does deliver—and what aggregate evaluation cannot—is a mechanistic account of each confirmed miss, down to the source line that causes it.

7 Implications and Future Work

For scanner evaluation. Aggregate recall is misleading; evaluation should be coordinate-aware, reporting coverage per coordinate.

For the skill ecosystem. Single-scanner governance inherits that scanner's blind spots wholesale. The coverage matrix quantifies the case for defense-in-depth: after sanitization, no single attack form evades all three scanners simultaneously, yet each scanner has a form on which its detection collapses ($\leq$27%). Complementary operational surfaces are not redundancy; they are coverage.

For adversarial methodology. Hypothesis locking separates genuine prediction from post-hoc rationalization and should become standard practice.

For scanner design. The confirmed blind spots are actionable as engineering fixes, and the decomposition says which kind each is. SkillSpector's dotfile skip is a coverage defect: include hidden files in the build context. Cisco's threshold swallowing is a reporting defect: surface MEDIUM findings instead of discarding them (80 of them in our corpus alone). The regex engine's no-literal blindness is an architectural limit, not a bug, which is precisely the information a coverage matrix, and only a coverage matrix, delivers.

For evaluation methodology. Three traps surfaced in our own measurements and had to be corrected before any conclusion was stable: threshold swallowing (80 MEDIUM findings silently discarded by a safe/unsafe threshold in the main experiment), technical failures indistinguishable from misses in raw logs, and ground-truth leakage through provenance metadata. Benchmarks that do not control for these mis-report detection.

Future work. We are extending this into a full benchmark and a taxonomy-guided defensive filter, to be reported in an extended version. Beyond agent skills, the same loop—observation language, decomposition, locked hypotheses, adversarial validation—applies wherever multiple automated analyses pass judgment on the same artifact and disagree: package registries, dependency scanners, static analyzers, LLM-based security evaluators. Every such setting already produces disagreement; what it lacks is the instrument to read it.

Takeaway. Disagreement among security analyses is usually filed as an evaluation inconvenience. Read structurally, it is a map of

Table 5: The prediction chain: structural evidence → locked blind-spot hypothesis → pre-locked construction → validation → mechanism. All hypotheses were locked on 2026-08-13 (coverage matrix + source reading); all constructions ran on 2026-08-19 or later. Read left to right: each row is one full traversal of the loop.

Structural evidence	Predicted blind spot	Pre-locked construction	Validation	Mechanism
SkillSpector's build context skips dotfiles	hidden-file payload unseen	payload entirely in `.env`/`.profile`/`.gitconfig`	**14/15** evade SS	file collection
Cisco requires literal command triggers	indirect execution unseen	indirect construction variants	**6/6** evade Cisco	trigger surface
Regex engine matches lexical markers only	semantic wording unseen	tool-semantics phrasing, no literals	**10/10** evade Cat	lexical matching
Independent checks, additive scoring	conjunction under-scored	stacked weak mechanisms	**5/5** partial bypass SS	score composition
Wild pipeline-injection precedent	runtime dataflow unseen	variable-injection pipelines (wild patterns)	**3/5** score 0 on SS	static dataflow

what each analysis cannot see, and that map can be drawn before an attacker walks it.

8 Related Work

Prior work addresses individual parts of this problem. Malicious-skill benchmarks [3, 9] provide corpora and taxonomies but not a shared cross-scanner measurement space: MalSkillBench labels a corpus along its own three-dimensional taxonomy for training and evaluating detectors, whereas we map *scanner outputs* onto a coordinate space to interrogate disagreement between scanners. Scanner evaluations [1, 2, 6, 7] quantify disagreement or recall but do not structurally attribute it; SkillsMetric maps the detection boundary of a single self-built static framework, while we analyze production scanners and predict their failures before constructing them. Adversarial-generation studies [5, 10] demonstrate that evasion is possible (SkillCloak's payload-preserving transformations bypass eight scanners at over 90%; ColluSkill's cross-skill chains evade six scanners at 96%) but search for evasions generically or attack the composition layer rather than single-skill architecture, whereas our constructions are derived from locked hypotheses about *where* each scanner's architecture fails, and our misses are traced to source-level mechanisms. Implementation analyses such as SkillProbe [4] formalize declaration-implementation inconsistency (over- and under-declaration) and combinatorial risk, but audit one framework's findings rather than connecting them to a cross-scanner attack model. Our contribution connects these steps into a single analysis loop.

References

[1] Xinze Chen, Chi Zhang, Ping Ji, and Yimin Liu. 2026. SkillsMetric: Mapping the Detection Boundary of Static Analysis for Malicious Agent Skills. *arXiv preprint arXiv:2608.08468* (2026).

[2] Bacem Etteib, Daniele Lunghi, and Tégawendé F. Bissyandé. 2026. Detecting Malicious Agent Skills in the Wild using Attention. *arXiv preprint arXiv:2606.23416* (2026).

[3] Wenbo Guo, Wei Zeng, Chengwei Liu, Xiaojun Jia, Yijia Xu, Lei Tang, Yong Fang, and Yang Liu. 2026. MalSkillBench: A Runtime-Verified Benchmark of Malicious Agent Skills. *arXiv preprint arXiv:2606.07131* (2026).

[4] Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. 2026. SkillProbe: Security Auditing for Emerging Agent Skill Marketplaces via Multi-Agent Collaboration. *arXiv preprint arXiv:2603.21019* (2026).

[5] Zimo Ji, Congying Xu, Zongjie Li, Yudong Gao, Xin Wei, Shuai Wang, and Shing-Chi Cheung. 2026. Cloak and Detonate: Scanner Evasion and Dynamic Detection of Agent Skill Malware. *arXiv preprint arXiv:2607.02357* (2026).

[6] Vincent Koc, Patrick Erichsen, Jacob Tomlinson, Agustin Rivera, Michael Appel, and Nir Paz. 2026. ClawHub Security Signals: When VirusTotal, Static Analysis, and SkillSpector Disagree. *arXiv preprint arXiv:2606.01494* (2026).

[7] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale. *arXiv preprint arXiv:2601.10338* (2026).

[8] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi, and Maksym Andriushchenko. 2026. Skill-Inject: Measuring Agent Vulnerability to Skill File Attacks. *arXiv preprint arXiv:2602.20156* (2026).

[9] Tencent Zhuque Lab and The Chinese University of Hong Kong. 2026. Skill-TrustBench: Benchmark for AI Agent Skill Security Scanners. https://matrix.tencent.com/skilltrustbench/.

[10] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColluSkill: Adversarial Cross-Skill Composition for Evading Agent Skill Scanners. *arXiv preprint arXiv:2608.09732* (2026).

[11] Chi Zhang, Yimin Liu, Xinze Chen, and Ping Ji. 2026. What Keeps Agent Skills from Being Reusable? Evidence from 138K SKILL.md Files. *arXiv preprint arXiv:2608.08453* (2026).